Credit Card Approval Prediction

Project Description

Financial institutions receive a large number of applications on a daily basis. Traditional approval methods often rely on manual verification and predefined rules, which can be time-consuming and susceptible to human errors. Leveraging machine learning techniques to analyze applicant information, this project aims to provide a data-driven approach to automate credit card approval prediction.

The primary objective of this project is to develop a machine learning model capable of accurately predicting whether a credit card application should be approved based on factors such as income level, employment status, existing loans, credit history, and financial behavior. This system will assist financial institutions in making faster, more consistent approval decisions while reducing credit-related risks.

Social Impact:

Promotes fair and efficient access to financial services by providing unbiased credit card approval predictions, reducing application processing time, and improving the overall customer experience.

Business Model /Impact:

The project offers significant benefits to banks, credit card providers, and financial institutions by streamlining the approval process. This could lead to reduced operational costs, improved risk assessment, enhanced customer satisfaction, and increased revenue through efficient and data-driven credit evaluation.

Existing Solutions:

Credit Card Approval

Recommended Technology Stacks: Python, Scikit-Learn, XGBoost, Random Forest, Pandas, NumPy, Flask, IBM Cloud

References:

https://www.sciencedirect.com/topics/computer-science/credit-scoring

Scenarios:-

1. Eligible applicant approved:-

An applicant with a stable income, sufficient work experience, and a good credit history applies for a credit card through the web application.

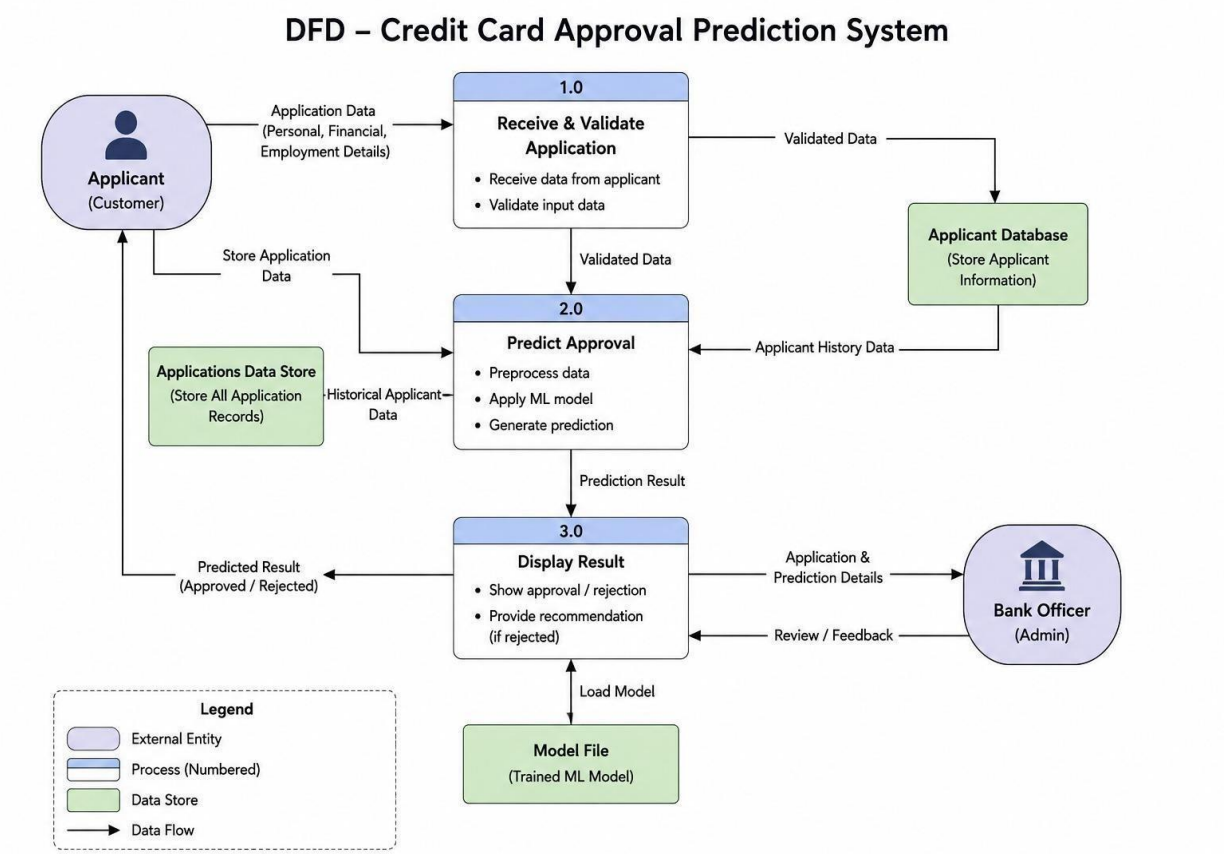
2. High-risk applicant rejected:-

An applicant with limited work experience, multiple dependents, and a short credit history submits a credit card application.

3. Borderline applicant evaluated objectively:-

An applicant has moderate income and employment history but an average credit history. The prediction depends on the combination of all features rather than a single factor.

DFD/Architecture:-



Prerequisites

1. Flask Framework Knowledge
2. HTML, CSS, and JavaScript Skills
3. Python Programming Proficiency 4. Version Control with Git
5. Development Environment Setup

Project Workflow

Milestone 1: Model Selection and Architecture

- Dataset collection
- Cleaning
- Encoding
- Model selection

Milestone 2: Core Functionalities Development

- Feature engineering
- Training
- Evaluation

Milestone 3: App.py Development

- Routes
- Load model

- Prediction

Milestone 4: Frontend Development

- Home
- Form
- Results

Milestone 5: Deployment

- GitHub
- Render • Testing

Milestone 6: Conclusion

•

Performance •
Future scope

Milestone 1: Model Selection and Architecture

Dataset Collection

The first step involved collecting the required datasets for building the credit card approval prediction model. Two datasets were used: **application_record.csv**, which contains applicants' personal, employment, and financial details, and **credit_record.csv**, which contains their historical credit repayment information. These datasets were merged using the applicant ID to create a comprehensive dataset suitable for machine learning.

Epic 1: Data collection

In [103]:

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/conte
nt/drive", force_remount=True).
```

In [104]:

```
credit = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/credit_record.csv')
app = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/application_record.csv')
```

Data Cleaning

Before training the model, the collected data was cleaned to improve its quality. Duplicate records were removed, missing values were handled appropriately, unnecessary columns such as mobile and phone flags were dropped, and negative values in age and employment duration were converted into positive values. Additional preprocessing, such as creating the credit history window and mapping the credit status values, ensured that the dataset was consistent and ready for analysis.

3)Data cleaning and merging:-

```
def data_cleaning(app, credit):
    # --- 1. CLEANING APPLICANT DATA ---

    # Create family_dependency: combine children and family members
    app['family_dependency'] = app['CNT_CHILDREN'] + app['CNT_FAM_MEMBERS']

    # Handle negative values for time-related features
    app['DAYS_BIRTH'] = abs(app['DAYS_BIRTH'])
    app['DAYS_EMPLOYED'] = abs(app['DAYS_EMPLOYED'])

    # Drop unnecessary columns (Example list)
    cols_to_drop = ['FLAG_MOBIL', 'FLAG_WORK_PHONE', 'FLAG_PHONE', 'FLAG_EMAIL']
    app.drop(columns=cols_to_drop, inplace=True, errors='ignore')

    # Map categorical columns to numerical labels
    # Example mapping for Housing Type
    mapping_dict = {'House / apartment': 0, 'With parents': 1, 'Municipal apartment': 2,
                    'Rented apartment': 3, 'Office apartment': 4, 'Co-op apartment': 5}
    app['NAME_HOUSING_TYPE'] = app['NAME_HOUSING_TYPE'].map(mapping_dict)

    # --- 2. CLEANING AND MERGING CREDIT DATA ---

    # Group by ID to handle multiple records
    # Create new features based on credit history period
    credit['open_month'] = credit.groupby('ID')['MONTHS_BALANCE'].transform('min')
    credit['end_months'] = credit.groupby('ID')['MONTHS_BALANCE'].transform('max')
    credit['window'] = credit['end_months'] - credit['open_month']

    # Interpret STATUS (Example logic: 0-5 are overdue, C/X are good/no records)
    status_map = {'C': 0, 'X': 0, '0': 1, '1': 2, '2': 3, '3': 4, '4': 5, '5': 6}
    credit['STATUS'] = credit['STATUS'].map(status_map)

    # Aggregate credit data by ID before merging with applicant data
    credit_agg = credit.groupby('ID').agg({
        'STATUS': 'max',
        'window': 'first'
    }).reset_index()

    # --- 3. MERGE ---
    final_df = pd.merge(app, credit_agg, on='ID', how='inner')

    return final_df

# Usage:
# cleaned_data = data_cleaning(application_record, credit_record)
```

Encoding

Machine learning models require numerical input, so all categorical attributes were converted into numeric values using **Label Encoding**. Features such as gender, income type, education level, family status, housing type, occupation type, vehicle ownership, and property ownership were encoded into numerical labels. This transformation enabled the Random Forest algorithm to process the categorical information efficiently while preserving all available features.

5)Handling categorical values:-

```
from sklearn.preprocessing import LabelEncoder

cg = LabelEncoder()
oc = LabelEncoder()
own_r = LabelEncoder()
it = LabelEncoder()
et = LabelEncoder()
fs = LabelEncoder()
ht = LabelEncoder()


final_df['CODE_GENDER'] = cg.fit_transform(final_df['CODE_GENDER'])
final_df['FLAG_OWN_CAR'] = oc.fit_transform(final_df['FLAG_OWN_CAR'])
final_df['FLAG_OWN_REALTY'] = own_r.fit_transform(final_df['FLAG_OWN_REALTY'])
final_df['NAME_INCOME_TYPE'] = it.fit_transform(final_df['NAME_INCOME_TYPE'])
final_df['NAME_EDUCATION_TYPE'] = et.fit_transform(final_df['NAME_EDUCATION_TYPE'])
final_df['NAME_FAMILY_STATUS'] = fs.fit_transform(final_df['NAME_FAMILY_STATUS'])
final_df['NAME_HOUSING_TYPE'] = ht.fit_transform(final_df['NAME_HOUSING_TYPE'])
```

Model Selection

Several machine learning algorithms were evaluated to identify the most suitable model for predicting credit card approval. Logistic Regression and Random Forest were compared using evaluation metrics such as accuracy, precision, recall, F1-score, and ROC-AUC score. Based on its superior predictive performance, ability to handle complex relationships, and better classification of both approved and rejected applicants, the **Random Forest Classifier** was selected as the final model for the project.

Milestone 2: Core Functionalities Development

Feature Engineering

Feature engineering was performed to improve the quality of the dataset and enhance the model's predictive performance. Relevant features such as applicant income, age, employment duration, education level, family status, housing type, occupation type, and credit history window were selected for training. The target variable (**STATUS_BIN**) was created by converting the original credit status into binary classes representing approved and rejected applicants. Irrelevant features were removed to ensure the model learned only from meaningful information.

```
Important Features
                Feature  Importance
13               window    0.223058
9             DAYS_BIRTH    0.182312
10        DAYS_EMPLOYED    0.155032
4       AMT_INCOME_TOTAL    0.126380
11       OCCUPATION_TYPE    0.076975
7     NAME_FAMILY_STATUS    0.034929
5       NAME_INCOME_TYPE    0.032917
12       CNT_FAM_MEMBERS    0.031782
6    NAME_EDUCATION_TYPE    0.030693
3           CNT_CHILDREN    0.023255
1           FLAG_OWN_CAR    0.022692
8      NAME_HOUSING_TYPE    0.021403
2        FLAG_OWN_REALTY    0.020087
0            CODE_GENDER    0.018485

Out-of-Bag Score : 0.8548
```

Training

The processed dataset was divided into training and testing sets using an 80:20 split. The Random Forest Classifier was trained using the training dataset, allowing it to learn patterns and relationships between applicant characteristics and credit approval outcomes. Appropriate model parameters such as the number of trees, maximum depth, minimum samples per split, and class balancing were configured to improve prediction accuracy and reduce overfitting.

```
X = final_df.drop(columns=['ID','STATUS','STATUS_BIN'])

# Target
y = final_df['STATUS_BIN']

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Evaluation

After training, the model was evaluated using the testing dataset to measure its prediction performance. Various evaluation metrics, including **Accuracy**, **Precision**, **Recall**, **F1-Score**, **ROC-AUC Score**, and the **Confusion Matrix**, were used to assess the model. The Random Forest model achieved an accuracy of approximately **86%** with a **ROC-AUC score of 0.78**, demonstrating its effectiveness in predicting credit card approval decisions. Feature importance analysis was also performed to identify the most influential factors affecting the model's predictions.

```
Initializing Random Forest Classifier...
Training Random Forest...
Generating Predictions...

==================================================
Random Forest Evaluation
==================================================

Accuracy : 0.8640
Precision : 0.4251
Recall    : 0.4429
F1 Score  : 0.4338

Confusion Matrix
[[5920  514]
 [ 478  380]]

Classification Report
              precision    recall  f1-score   support

           0       0.93      0.92      0.92      6434
           1       0.43      0.44      0.43       858

    accuracy                           0.86      7292
   macro avg       0.68      0.68      0.68      7292
weighted avg       0.87      0.86      0.87      7292

ROC-AUC Score : 0.7822
```

Milestone 3: App.py Development

Routes

The Flask application was developed using multiple routes to manage different pages and user interactions. The home route displays the welcome page, the prediction route provides the applicant input form, and the result route receives the submitted data, processes it, and displays the prediction outcome. These routes ensure smooth navigation and communication between the user interface and the machine learning model. **Load Model**

The trained **Random Forest** model was saved as a **credit_card_model.pkl** file using the Joblib library. During application startup, this saved model is loaded into memory within the Flask application. Loading the pre-trained model eliminates the need to retrain the model each time the application runs, resulting in faster predictions and improved efficiency.

```
import joblib

joblib.dump(rf_model, "credit_card_model.pkl")

print("Model Saved Successfully!")
```

Prediction

When the user submits the application form, the entered details are collected and converted into the same format used during model training. The Flask application creates a DataFrame with the input features and passes it to the loaded Random Forest model for prediction. Based on the model's output, the system displays either **"Credit Card Approved"** or **"Credit Card Rejected"** on the result page, providing users with an instant prediction.

```
from flask import Flask, render_template,
request import pandas as pd import joblib

app = Flask(__name__)
# ============================
# Load Trained Model
# ============================ model =
joblib.load("credit_card_model.pkl")

# If you used StandardScaler during training,
# uncomment the next line and use scaler.transform() # scaler
= joblib.load("scaler.pkl")



# ============================
# Home Page
# ============================
@app.route("/")
def home():
    return render_template("home.html")

# ============================
# Prediction Form
#
============================
@app.route("/predict") def
predict():
    return render_template("index.html")

# ============================ #
Result
# ============================
```

```
@app.route("/result", methods=["POST"]) def
result():

    try:
        # ----------------------------
# Collect Input Values
        # ----------------------------        gender =
int(request.form["gender"])        own_car =
int(request.form["own_car"])        own_realty =
int(request.form["own_realty"])        income =
float(request.form["income"])        income_type =
int(request.form["income_type"])        education =
int(request.form["education"])        family_status =
int(request.form["family_status"])        housing_type =
int(request.form["housing_type"])        occupation =
int(request.form["occupation"])        age =
float(request.form["age"])        experience =
float(request.form["experience"])        # Convert to the
format expected by the model        days_birth = age *
365        if income_type == 1:        # Pensioner
days_employed = 365243        else:
            days_employed = experience * 365        children
= int(request.form["children"])        family_members =
float(request.form["family_members"])        credit_window =
float(request.form["window"])

        # ------------------------------------
        # Create DataFrame
        # Feature order MUST match training
        # ------------------------------------

        data =
pd.DataFrame([[
gender,            own_car,
own_realty,
children,            income,
income_type,
education,
family_status,
housing_type,
days_birth,
days_employed,
occupation,
family_members,
credit_window
        ]],columns=[
```

```
            "CODE_GENDER",
            "FLAG_OWN_CAR",
            "FLAG_OWN_REALTY",
            "CNT_CHILDREN",
            "AMT_INCOME_TOTAL",
            "NAME_INCOME_TYPE",
            "NAME_EDUCATION_TYPE",
            "NAME_FAMILY_STATUS",
            "NAME_HOUSING_TYPE",
            "DAYS_BIRTH",
            "DAYS_EMPLOYED",
            "OCCUPATION_TYPE",
            "CNT_FAM_MEMBERS",
            "window"
        ])

        # --------------------------
        # Scale Data if Required
        # --------------------------

        # data = scaler.transform(data)

        prediction = model.predict(data)[0]

        if prediction == 1:
            prediction_text = "Credit Card Approved"
status = "approved"         else:
            prediction_text    =    "Credit    Card    Rejected"
status = "rejected"

        return render_template(
"result.html",
prediction=prediction_text,
status=status
        )

    except Exception as e:         return
render_template(             "result.html",
prediction="Error : " + str(e),
status="error"
        )

# ============================
# Run Flask
# ============================

if __name__ == "__main__":
```

```
app.run(debug=True)
```

Milestone 4: Frontend Development

Home Page

The Home page serves as the entry point of the Credit Card Approval Prediction System. It provides a clean and responsive user interface with the project title, a brief introduction, and navigation options. The page is designed using **HTML, CSS, and Bootstrap** to create an attractive layout and guide users to the prediction module.

Prediction Form

The Prediction Form was developed to collect all the applicant details required for credit card approval prediction. Users enter information such as gender, annual income, education level, occupation type, age, work experience, family details, and credit history. Form validation is implemented to ensure that all mandatory fields are completed before submission, improving data accuracy and user experience.

Result Page

The Result page displays the prediction generated by the trained Random Forest model. After the user submits the application form, the system processes the input data and presents the outcome as either **"Credit Card Approved"** or **"Credit Card Rejected."** The page also includes appropriate icons, messages, and navigation buttons, allowing users to perform another prediction or return to the Home page easily.

Milestone 5: Deployment

GitHub

- Created a GitHub repository for the project.
- Uploaded the complete source code, including Flask application, templates, static files, and documentation.
- Managed project versions using Git for easy collaboration and maintenance.
- My repository link is :- https://github.com/samianushna-cmyk/credit_card_approval/tree/main/Credit_card_approval_system-main

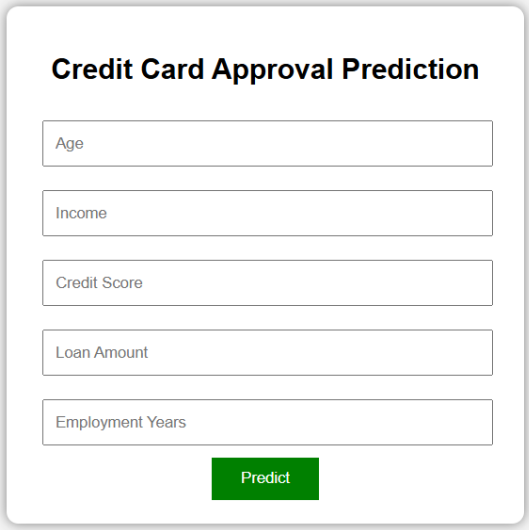
Render

- Connected the GitHub repository to the Render cloud platform.
- Configured the deployment settings, including `requirements.txt`, `Procfile`, and Python runtime.
- Deployed the Flask application to generate a publicly accessible web application.
- Deployment link is:- https://smartbridge-credit-card-approval.onrender.com/

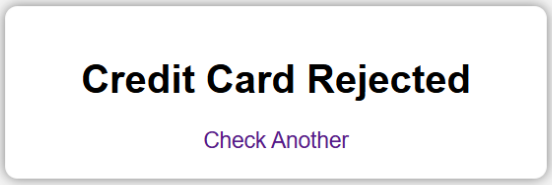
Testing

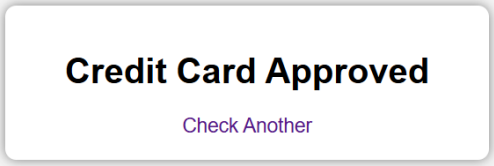
- Verified that the application loads correctly after deployment.
- Tested the prediction form using different applicant details.
- Confirmed that the model returns accurate approval/rejection results.
- Checked navigation between Home, Prediction, and Result pages.
- Ensured the application handled invalid inputs and runtime errors gracefully. **Exploring the Website's Web Pages:**

Input Section:



Results Section:





Milestone 6: Conclusion

The **Credit Card Approval Prediction System** successfully demonstrates the application of machine learning in automating the credit card approval process. By utilizing historical applicant and credit record data, the system predicts whether an applicant is likely to be approved or rejected based on various financial, demographic, and employment-related features. Data preprocessing, feature engineering, and label encoding were performed to prepare the dataset, and multiple machine learning algorithms were evaluated. Among them, the **Random Forest Classifier** achieved the best performance, with an accuracy of approximately **86%** and a **ROC-AUC score of 0.78**, making it the final model for deployment.

The trained model was integrated into a **Flask-based web application**, allowing users to enter applicant details through an intuitive interface and receive instant approval predictions. The application was thoroughly tested to ensure accurate predictions, smooth navigation, and reliable performance. Overall, the project provides a practical, efficient, and scalable solution for assisting financial institutions in making faster and more consistent credit card approval decisions while reducing manual effort. With future enhancements such as real-time credit bureau integration, explainable AI, database support, and advanced machine learning models, the system can be further improved for real-world deployment and decision support.